

PYTHON PARA APLICAÇÕES REAIS

COMO PYTHON É UTILIZADO NO MERCADO



Caroline Perez Gonçalves
2026

PYTHON PARA APLICAÇÕES REAIS

Caroline Perez Gonçalves

SUMÁRIO

APRESENTAÇÃO.....	1
INTRODUÇÃO	2
MANIPULAÇÃO DE ARQUIVOS	3
ARQUIVOS TXT	5
ARQUIVOS CSV.....	6
ARQUIVOS JSON	8
DATAS E HORÁRIOS	10
EXPRESSÕES REGULARES	12
CONSUMO DE APIs.....	15
INTRODUÇÃO À ORIENTAÇÃO A OBJETOS.....	17
PROJETO PRÁTICO	19
AGRADECIMENTO.....	23
CONTATO.....	24

APRESENTAÇÃO

Aos que não me conhecem, meu nome é Caroline. Sou formada em Análise e Desenvolvimento de Sistemas, pós-graduada em Inteligência Artificial e Machine Learning e possuo MBA em Gerenciamento de Projetos.

Além da minha experiência profissional com tecnologia, projetos e equipes multidisciplinares, também atuo como instrutora. Sempre acreditei que o conhecimento deve ser compartilhado de forma simples, leve e acessível, e foi com esse propósito que esta apostila foi desenvolvida.

O intuito deste material é mostrar como o Python pode ser utilizado para resolver problemas reais do dia a dia. Muito mais do que aprender novos comandos, o objetivo agora é compreender como a linguagem é aplicada no mercado, automatizando tarefas, manipulando arquivos, consumindo informações de outros sistemas e desenvolvendo soluções que fazem parte da rotina de empresas e profissionais de diversas áreas.

Ao final desta apostila, espero que você se sinta mais preparado para criar aplicações úteis e perceba que o Python vai muito além dos exercícios de programação. A partir daqui, você começará a utilizar a linguagem para resolver desafios cada vez mais próximos da realidade.

Vamos aos estudos! Mas, antes, sente-se, relaxe e sinta-se em casa. Quero que este material continue sendo como uma boa conversa entre amigos, tornando o aprendizado mais leve, agradável e, principalmente, significativo.

Boa sorte e muito sucesso!

INTRODUÇÃO

Até aqui, você aprendeu os principais fundamentos do Python. Criou seus primeiros programas, trabalhou com estruturas condicionais, laços de repetição, funções, módulos e começou a organizar seus projetos de uma forma muito mais profissional.

Mas existe uma pergunta que costuma surgir nesse momento.

"Tudo isso é muito interessante... mas onde o Python é utilizado na prática?"

A resposta para essa pergunta tem um segredo.

Muita gente acredita que o Python serve apenas para criar programas ou que ele é utilizado exclusivamente por desenvolvedores. Mas a verdade é bem diferente.

Como assim, Carol?

O Python está presente em diversas áreas do mercado. Ele é utilizado para automatizar tarefas, manipular arquivos, analisar dados, consumir informações de outros sistemas, desenvolver aplicações Web, criar soluções para Inteligência Artificial e muito mais.

Quando você entende isso, percebe que aprender Python vai muito além de conhecer comandos da linguagem. Significa adquirir uma ferramenta capaz de resolver problemas reais, economizar tempo e facilitar atividades que antes seriam feitas manualmente.

E tem mais: no mercado de trabalho, dificilmente um desenvolvedor cria tudo do zero. Grande parte do trabalho consiste em utilizar bibliotecas, integrar sistemas e aproveitar recursos já existentes para construir soluções de forma mais rápida e eficiente.

É exatamente isso que começaremos a fazer nesta apostila. Vamos deixar um pouco os exemplos didáticos e desenvolver aplicações cada vez mais próximas das que encontramos no dia a dia de empresas e profissionais da área de tecnologia.

Para que esta introdução não fique muito extensa, vamos explorar cada assunto no momento certo. E, como todo bom aprendizado, vamos começar pelo começo.

Vamos lá.

MANIPULAÇÃO DE ARQUIVOS

Todos os programas que desenvolvemos tinham uma característica em comum: eles funcionavam apenas enquanto estavam em execução. Assim que o programa era encerrado, todas as informações eram perdidas. Mas deixa eu te fazer uma pergunta...

Imagine que você criou um sistema para cadastrar clientes. Durante o dia, você registra centenas de pessoas, fecha o programa e vai embora. No dia seguinte, ao abrir o sistema novamente, percebe que todos os cadastros desapareceram. Não seria nada legal, concorda?

É justamente para resolver esse problema que existem os arquivos. Eles permitem armazenar informações de forma permanente, fazendo com que os dados continuem disponíveis mesmo depois que o programa é encerrado. Pode parecer um detalhe, mas praticamente todos os sistemas que utilizamos no dia a dia trabalham com algum tipo de arquivo.

Na verdade, você convive com arquivos o tempo inteiro, mesmo sem perceber. Quando salva um documento, cria uma planilha, baixa uma foto, envia um currículo por e-mail ou faz o download de uma música, está trabalhando com arquivos.

O Python também consegue fazer isso.

Podemos criar novos arquivos, abrir arquivos já existentes, escrever informações, ler conteúdos e até modificar dados salvos anteriormente.

Vamos começar pelo básico. Para abrir um arquivo utilizamos a função `open()`. Imagine que desejamos criar um arquivo chamado **clientes.txt**.

```
arquivo = open("clientes.txt", "w")
```

Calma... vamos entender essa linha antes de continuar.

O primeiro parâmetro informa o nome do arquivo que desejamos abrir ou criar. Já o segundo parâmetro indica o modo de abertura. Nesse exemplo utilizamos "w", que vem da palavra inglesa *write* e significa **escrita**. Em outras palavras, estamos dizendo ao Python: "Abra esse arquivo porque quero gravar informações nele."

Depois de abrir o arquivo, podemos escrever seu conteúdo utilizando o método `write()`.

```
arquivo = open("clientes.txt", "w")
```

```
arquivo.write("Caroline")
```

```
arquivo.close()
```

Percebeu que apareceu uma instrução nova? O método `close()` serve para fechar o arquivo quando terminamos de utilizá-lo. É uma boa prática fazer isso, pois garante que todas as informações sejam gravadas corretamente.

Agora imagine a situação inversa. O arquivo já existe e você deseja visualizar seu conteúdo. Nesse caso, utilizamos o modo "r", que significa *read*, ou seja, leitura.

```
arquivo = open("clientes.txt", "r")  
  
conteudo = arquivo.read()  
  
print(conteudo)  
  
arquivo.close()
```

Se o arquivo possuir o nome **Caroline**, esse será exatamente o conteúdo exibido na tela. Simples, não é?

Mas existe outra situação bastante comum. Imagine um cadastro de clientes. Sempre que um novo cliente é registrado, não queremos apagar os dados anteriores. Queremos apenas acrescentar uma nova informação ao final do arquivo. Para isso utilizamos o modo "a", de *append*.

```
arquivo = open("clientes.txt", "a")  
  
arquivo.write("\nJoão")  
  
arquivo.close()
```

Nesse caso, o novo conteúdo será adicionado ao final do arquivo, preservando todas as informações que já estavam salvas.

Até aqui aprendemos uma forma que funciona muito bem. Porém, existe uma maneira ainda mais recomendada pelos desenvolvedores. Ela utiliza a palavra-chave `with`.

`with open("clientes.txt", "r") as arquivo:`

```
    conteudo = arquivo.read()  
  
    print(conteudo)
```

Quando utilizamos `with`, o próprio Python se encarrega de fechar o arquivo automaticamente ao final da execução. Isso torna o código mais seguro e evita problemas caso esqueçamos de chamar o método `close()`. Sempre que possível, procure utilizar essa abordagem.

Agora pense em quantos sistemas dependem de arquivos para funcionar.

Uma clínica médica precisa armazenar prontuários. Uma escola salva informações dos alunos. Uma loja registra seus produtos e vendas. Um escritório mantém contratos e documentos importantes. Todos esses sistemas precisam guardar informações para que elas continuem existindo mesmo depois que o programa é encerrado.

Resumindo, o que eu quero que você aprenda aqui, é que arquivos permitem armazenar dados de forma permanente e que o Python oferece recursos para criar, ler e modificar esses arquivos. Também vimos os principais modos de abertura: "r" para leitura, "w" para escrita e "a" para adicionar informações ao final do arquivo. Além disso, vimos que a estrutura `with` é a forma mais recomendada de trabalhar com arquivos, pois o próprio Python realiza o fechamento automaticamente.

Até aqui, nossos programas guardavam informações apenas na memória. A partir deste momento, eles passam a registrar dados de verdade, permitindo que as informações permaneçam disponíveis mesmo depois que a aplicação é encerrada. E pode acreditar... esse é um dos recursos mais utilizados no desenvolvimento de sistemas.

ARQUIVOS TXT

Depois de aprender como abrir, criar e modificar arquivos utilizando Python, chegou a hora de conhecer alguns formatos muito comuns no dia a dia. E vamos começar pelo mais simples deles: o arquivo **TXT**.

Muito provavelmente você já abriu um arquivo .txt alguma vez. Ele pode ser criado no Bloco de Notas do Windows, no TextEdit do macOS ou em qualquer editor de texto. Diferentemente de documentos como Word ou PDF, um arquivo TXT armazena apenas texto, sem imagens, tabelas, cores ou formatações especiais.

Pode parecer algo simples demais, mas não se engane. Arquivos TXT continuam sendo muito utilizados em sistemas para armazenar configurações, registrar informações, criar arquivos de log e até trocar dados entre aplicações.

Pense que um sistema que registra tudo o que acontece durante sua execução. Cada acesso realizado por um usuário pode ser gravado em um arquivo TXT para futuras consultas. Se algum problema acontecer, basta abrir esse arquivo e verificar o histórico das operações realizadas.

Criar um arquivo TXT é exatamente igual ao que vimos no capítulo anterior.

with open("mensagem.txt", "w") as arquivo:

```
    arquivo.write("Olá! Seja bem-vindo ao cursinho de Python.")
```

Ao executar esse código, será criado um arquivo chamado **mensagem.txt**, contendo o texto informado.

Agora imagine que você queira visualizar esse conteúdo.

with open("mensagem.txt", "r") as arquivo:

```
    conteudo = arquivo.read()
```

```
print(conteudo)
```

Resultado: Olá! Seja bem-vindo ao cursinho de Python.

Até aqui nenhuma novidade. Mas existe um recurso muito interessante. Imagine que o arquivo possua várias linhas.

Caroline

João

Fernanda

Carlos

Podemos percorrer esse arquivo linha por linha utilizando um laço for.

with open("clientes.txt", "r") as arquivo:

for linha in arquivo:

print(linha)

Percebe como isso é útil?

Em vez de ler todo o arquivo de uma única vez, podemos processar cada linha separadamente. Esse recurso é bastante utilizado quando trabalhamos com arquivos grandes.

Também podemos adicionar novas informações sem apagar o conteúdo existente.

with open("clientes.txt", "a") as arquivo:

arquivo.write("\nMariana")

Observe que utilizamos o modo "a", responsável por acrescentar novos dados ao final do arquivo.

Os arquivos TXT costumam ser utilizados quando precisamos armazenar informações simples, como listas de nomes, registros de atividades, configurações de um programa ou pequenos relatórios. Apesar de sua simplicidade, continuam sendo uma excelente opção em diversas situações. Eles são rápidos, leves e muito utilizados para registrar informações que não exigem uma estrutura mais complexa.

ARQUIVOS CSV

Uma empresa exportou a lista de todos os seus clientes. Quando você abre o arquivo, encontra algo parecido com isto:

Nome,Idade,Cidade

Ana,25,Curitiba

Carlos,31,São Paulo

Fernanda,28,Belo Horizonte

À primeira vista, parece apenas um arquivo de texto.

Mas existe uma diferença importante.

Cada informação está organizada em “colunas”, assim que abrimos o arquivo em programas como o excel.

Esse formato recebe o nome de **CSV**, sigla para *Comma-Separated Values*, ou, em português, **Valores Separados por Vírgula**.

Apesar do nome, nem sempre o separador será uma vírgula. Dependendo da configuração do sistema ou da região, também é muito comum encontrar arquivos separados por ponto e vírgula (;).

Pode acreditar.

Você provavelmente já utilizou um arquivo CSV sem perceber.

Quando exportamos uma planilha do Excel, baixamos um relatório de vendas, importamos uma lista de contatos ou transferimos informações entre dois sistemas diferentes, existe uma grande chance de estarmos trabalhando com um arquivo CSV. Vamos criar um exemplo:

```
with open("clientes.csv", "w") as arquivo:
```

```
    arquivo.write("Nome,Idade,Cidade\n")
```

```
    arquivo.write("Ana,25,Curitiba\n")
```

```
    arquivo.write("Carlos,31,São Paulo")
```

O conteúdo do arquivo será:

Nome,Idade,Cidade

Ana,25,Curitiba

Carlos,31,São Paulo

Agora imagine que desejamos ler esse arquivo.

```
with open("clientes.csv", "r") as arquivo:
```

```
    for linha in arquivo:
```

```
        print(linha)
```

Resultado:

Nome,Idade,Cidade

Ana,25,Curitiba

Carlos,31,São Paulo

Percebeu como é parecido com um arquivo TXT?

A principal diferença está na organização das informações. No CSV, cada linha representa um registro, enquanto cada coluna representa um dado específico daquele registro.

É justamente essa organização que faz com que esse formato seja tão utilizado por empresas.

Imagine uma loja com dez mil produtos cadastrados. Exportar essas informações para um arquivo CSV permite abrir os dados em programas como Excel, LibreOffice Calc ou importar facilmente para outro sistema.

Na prática, arquivos CSV são utilizados para exportação de planilhas, relatórios financeiros, cadastro de clientes, controle de estoque, importação e exportação de dados entre sistemas.

Se vale um resumo, eu diria que os arquivos CSV são uma forma simples e eficiente de organizar grandes quantidades de informações em linhas e colunas. Por isso, estão entre os formatos mais utilizados no mercado quando o objetivo é compartilhar ou importar dados entre diferentes aplicações.

ARQUIVOS JSON

Conhecemos dois formatos bastante utilizados para armazenar informações: o TXT e o CSV. Ambos são excelentes para diversas situações, mas existe um formato que ganhou enorme destaque nos últimos anos e que provavelmente fará parte da sua rotina como desenvolvedor: o **JSON**.

Calma... esse nome pode parecer complicado, mas a ideia é bem simples.

JSON é a sigla para **JavaScript Object Notation**. Apesar do nome fazer referência ao JavaScript, esse formato é utilizado por praticamente todas as linguagens de programação, incluindo o Python.

Imagine que você precise representar um cliente com várias informações.

Nome.

Idade.

Cidade.

Telefone.

Como armazenar tudo isso em um arquivo de forma organizada?

É justamente para isso que o JSON foi criado. Olha só um exemplo:

```
{  
    "nome": "Caroline",  
    "idade": 32,  
    "cidade": "São Paulo"  
}
```

Percebeu alguma semelhança com algo que vimos?

Se você respondeu "dicionário", acertou.

Na verdade, um objeto JSON se parece muito com um dicionário do Python. Ambos organizam as informações utilizando pares de **chave** e **valor**.

É por isso que trabalhar com JSON costuma ser tão natural para quem já conhece dicionários.

Lendo um arquivo JSON

Para trabalhar com esse formato utilizamos a biblioteca padrão json.

Vamos imaginar que exista um arquivo chamado **cliente.json**.

```
{  
    "nome": "Caroline",  
    "idade": 32,  
    "cidade": "São Paulo"  
}
```

Agora podemos ler esse arquivo utilizando Python.

```
import json  
  
with open("cliente.json", "r") as arquivo:  
    cliente = json.load(arquivo)  
  
print(cliente)
```

Resultado: {'nome': 'Caroline', 'idade': 32, 'cidade': 'São Paulo'}

Observe que o conteúdo do arquivo foi convertido para um dicionário. Isso significa que podemos acessar qualquer informação normalmente.

```
print(cliente["nome"])
```

Resultado: Caroline

Gravando informações em JSON

Também podemos criar arquivos JSON utilizando Python. Veja um exemplo:

```
import json

cliente = {
    "nome": "João",
    "idade": 28,
    "cidade": "Curitiba"
}

with open("cliente.json", "w") as arquivo:

    json.dump(cliente, arquivo, indent=4)
```

O parâmetro indent=4 serve apenas para deixar o arquivo mais organizado e fácil de ler.

Onde o JSON é utilizado?

Pode acreditar... Em praticamente todos os lugares.

Quando um aplicativo consulta informações em um servidor.

Quando um site envia dados para outro sistema.

Quando uma aplicação conversa com uma API.

Quando um serviço em nuvem troca informações.

Muito provavelmente existe um JSON envolvido.

Ele se tornou um padrão justamente porque é simples, organizado e muito fácil de interpretar tanto por pessoas quanto por programas.

Você verá JSON inúmeras vezes durante sua carreira como desenvolvedor.

DATAS E HORÁRIOS

Você desenvolveu um sistema de vendas.

Sempre que um cliente realiza uma compra, deseja registrar exatamente quando aquela operação aconteceu. Ou imagine um sistema de controle de acesso.

Como descobrir em qual dia e horário um funcionário entrou na empresa? Essas são situações muito comuns.

E, felizmente, o Python já possui ferramentas prontas para resolvê-las.

Como vimos na apostila anterior, existe uma biblioteca chamada `datetime`, responsável por trabalhar com datas e horários.

Vamos conhecer alguns de seus principais recursos.

Obtendo a data e o horário atuais

Para descobrir a data e a hora atuais, utilizamos o método `now()`.

```
from datetime import datetime  
  
agora = datetime.now()  
  
print(agora)
```

Resultado: 2026-08-10 14:30:18

É importante lembrar que o valor exibido dependerá do momento em que o programa for executado.

Exibindo apenas a data

Nem sempre precisamos mostrar a data e a hora juntas.

Também podemos obter apenas a data atual.

```
from datetime import date  
  
hoje = date.today()  
  
print(hoje)
```

Resultado: 2026-08-10

Formatando datas

Talvez você tenha percebido que o formato exibido pelo Python não seja exatamente o mais comum para nós.

No Brasil, normalmente escrevemos a data assim: 10/08/2026

Para isso utilizamos o método `strftime()`.

```
from datetime import datetime  
agora = datetime.now()  
print(agora.strftime("%d/%m/%Y"))
```

Resultado: 10/08/2026

Também podemos exibir apenas o horário.

```
print(agora.strftime("%H:%M:%S"))
```

Resultado: 14:30:18

Esses formatos são muito utilizados em sistemas profissionais.

Onde utilizamos datas?

Muito mais do que você imagina. Sempre que um sistema precisa registrar quando alguma ação aconteceu, existe uma data ou um horário envolvido.

- Cadastro de clientes;
- Emissão de notas fiscais;
- Registro de vendas;
- Controle de estoque;
- Agendamentos;
- Controle de ponto;
- Histórico de acessos;
- Relatórios.

Imagine um banco.

Seria impossível registrar uma transferência sem informar quando ela foi realizada.

O mesmo acontece com praticamente qualquer sistema moderno.

EXPRESSÕES REGULARES

Se eu te perguntasse o que são **Expressões Regulares**, provavelmente esse nome pareceria um pouco assustador e você diria que não sabe o que é.

Mas, existe uma grande chance de você já utilizar esse conceito no dia a dia, mesmo sem saber.

Imagine um site em que você se cadastrou... Você informa seu e-mail.

O sistema imediatamente avisa se ele está em um formato válido.

Ou pense em um formulário que aceita apenas números no campo do CPF.

Ou ainda em um aplicativo que verifica se uma senha possui letras, números e caracteres especiais.

Tudo isso pode ser feito utilizando **Expressões Regulares**, também conhecidas como **Regex**.

Em poucas palavras, uma expressão regular é um padrão utilizado para localizar ou validar informações dentro de um texto.

Pode parecer complicado agora, mas daqui a pouco tudo vai fazer sentido.

A biblioteca re

No Python, trabalhamos com expressões regulares utilizando a biblioteca padrão re. Ela já acompanha a instalação da linguagem, portanto não precisamos instalar nada.

Vamos começar com um exemplo bem simples.

Imagine que queremos verificar se a palavra **Python** aparece em uma frase.

```
import re
```

```
texto = "Estou estudando Python."
```

```
resultado = re.search("Python", texto)
```

```
print(resultado)
```

Se a palavra for encontrada, o Python retornará um objeto indicando que ela existe naquele texto.

Caso contrário, o resultado será None.

Procurando padrões

Agora imagine uma empresa que possui milhares de documentos. Você precisa localizar todos aqueles que contenham a palavra **cliente**.

Seria inviável abrir arquivo por arquivo manualmente. Com uma expressão regular isso pode ser feito automaticamente.


```
import re

texto = "O cliente realizou a compra."

resultado = re.search("cliente", texto)

if resultado:

    print("Palavra encontrada.")
```

Resultado: Palavra encontrada.

Viu? O Python pesquisou o texto e encontrou exatamente o padrão informado.

Validando informações

Esse é, provavelmente, o uso mais comum das expressões regulares. Imagine que seu sistema peça um e-mail.

Antes de salvar esse cadastro, você deseja verificar se o formato parece válido.

```
import re

email = "caroline@email.com"

padrao = r".+@.+\.+"

if re.fullmatch(padrao, email):

    print("E-mail válido.")

else:

    print("E-mail inválido.")
```

Calma...

Você não precisa decorar esse padrão. Na verdade, quase ninguém faz isso. O importante agora é entender a ideia.

A expressão regular funciona como uma regra.

Ela verifica se o texto informado segue um determinado formato.

Encontrando vários resultados

Também podemos localizar todas as ocorrências de um padrão. Veja este exemplo:

```
import re

texto = "Python é incrível. Python é muito utilizado."

resultado = re.findall("Python", texto)

print(resultado)
```

Resultado: ['Python', 'Python']

Nesse caso, a função `findall()` retornou todas as vezes que a palavra apareceu no texto.

CONSUMO DE APIs

Até aqui, todos os programas que desenvolvemos trabalharam apenas com informações armazenadas no próprio computador. Mas pense que você esteja desenvolvendo um aplicativo de previsão do tempo. Como descobrir a temperatura da sua cidade? Ou imagine um sistema que precisa consultar a cotação do dólar em tempo real. Será que precisamos digitar essas informações manualmente todos os dias?

Claro que não.

Esses dados já existem em outros sistemas. O que precisamos é de uma forma de acessá-los. É justamente aí que entram as **APIs**.

Imagine um restaurante. Você faz um pedido ao garçom, ele leva a solicitação até a cozinha e, alguns minutos depois, retorna com o prato solicitado. Você nunca entra na cozinha para preparar a comida. Apenas faz um pedido e recebe uma resposta. Com uma API acontece exatamente a mesma coisa. Nosso programa envia uma solicitação para outro sistema e recebe uma resposta contendo as informações desejadas. Simples assim.

API é a sigla para **Application Programming Interface**, ou Interface de Programação de Aplicações.

Apesar do nome parecer complicado, a ideia é bastante simples. Ela funciona como uma ponte de comunicação entre diferentes sistemas. Graças às APIs, aplicativos conseguem consultar informações atualizadas sem precisar armazenar tudo localmente. É por isso que conseguimos ver a previsão do tempo, acompanhar um pedido de entrega, consultar a cotação de uma moeda ou acessar informações de um mapa em tempo real.

No Python, uma das bibliotecas mais utilizadas para consumir APIs é a **requests**.

E aqui vale uma observação importante: **a biblioteca requests não faz parte da biblioteca padrão do Python**. Isso significa que, antes de utilizá-la, normalmente será necessário instalá-la utilizando o gerenciador de pacotes `pip`, assunto que estudamos na apostila anterior. O comando é bem simples.

```
pip install requests
```

Depois da instalação, já podemos utilizá-la normalmente.

```
import requests

resposta = requests.get("https://api.exemplo.com/clientes")

print(resposta.status_code)
```

Estamos utilizando a função `get()` para enviar uma solicitação a um servidor. Em seguida, verificamos o código da resposta para saber se a comunicação foi realizada com sucesso.

Na maioria das vezes, as APIs devolvem as informações no formato **JSON**. Lembra dele? Acabamos de estudá-lo em assuntos anteriores. Um exemplo de resposta seria:

```
{
    "nome": "Caroline",
    "cidade": "São Paulo"
}
```

No Python, podemos converter essa resposta para um dicionário utilizando o método `json()`.

```
import requests

resposta = requests.get("https://api.exemplo.com/cliente")

dados = resposta.json()

print(dados["nome"])
```

Resultado: Caroline

Viu como os assuntos começam a se conectar? Primeiro aprendemos dicionários. Depois conhecemos os arquivos JSON. Agora estamos utilizando exatamente esses mesmos conceitos para receber informações enviadas por outro sistema. Nada do que estudamos até aqui foi por acaso.

As APIs estão presentes em praticamente todas as aplicações modernas. Sempre que um aplicativo busca informações na internet, existe uma grande chance de que ele esteja utilizando uma API. É assim que funcionam aplicativos de previsão do tempo, sistemas bancários, lojas virtuais, plataformas de pagamento, aplicativos de transporte, redes sociais e até diversas ferramentas de Inteligência Artificial.

Imagine, por exemplo, um aplicativo de entrega de comida. Quando você abre o aplicativo, ele exibe restaurantes próximos, cardápios, preços e tempo estimado de entrega.

Você acha que todas essas informações estão armazenadas no seu celular? Não.

O aplicativo faz diversas solicitações para servidores por meio de APIs e recebe tudo isso em poucos segundos. É exatamente esse tipo de comunicação que o Python também consegue realizar.

Um resumo que vale anotar: uma API permite que diferentes sistemas conversem entre si, trocando informações de maneira rápida e segura.

No Python, uma das bibliotecas mais utilizadas para esse tipo de comunicação é a `requests`, que, por não fazer parte da biblioteca padrão da linguagem, normalmente precisa ser instalada utilizando o `pip`. Depois disso, basta enviar uma solicitação para receber informações que poderão ser utilizadas normalmente dentro do seu programa.

Até aqui, nossos projetos trabalhavam apenas com informações criadas por nós. A partir deste momento, eles passam a buscar dados em outros sistemas e a interagir com serviços disponíveis na internet. Esse é um passo muito importante para quem deseja desenvolver aplicações modernas e mais próximas da realidade do mercado.

INTRODUÇÃO À ORIENTAÇÃO A OBJETOS

Aqui não vamos aprender tudo sobre POO, mas é legal saber como é.

Ao longo desta trilha aprendemos muitas coisas. Criamos variáveis, utilizamos estruturas condicionais, laços de repetição, funções, módulos e começamos a desenvolver aplicações cada vez mais próximas das utilizadas no mercado. Mas existe uma pergunta que costuma surgir quando os projetos começam a crescer.

Como organizar um sistema que possui milhares de linhas de código?

Imagine um sistema de uma concessionária. Existem clientes, veículos, vendedores, financiamentos, notas fiscais e diversas outras informações. Será que faz sentido guardar tudo isso em listas e dicionários espalhados pelo programa?

Funciona? Funciona.

Mas conforme o sistema cresce, essa organização começa a ficar cada vez mais difícil.

É justamente para resolver esse problema que existe a **Orientação a Objetos**, também conhecida pela sigla **POO**.

Pense em um carro. Todo carro possui algumas características.

Cor.

Marca.

Modelo.

Ano.

Placa.

Mas ele também possui comportamentos.

Pode ligar.

Frear.

Acelerar.

Virar.

Na programação podemos representar exatamente essa ideia.

Criamos um objeto chamado **Carro**, que reúne tanto as informações quanto as ações relacionadas a ele. É isso que a Orientação a Objetos faz.

Ela organiza o programa utilizando objetos que representam elementos do mundo real.

Pode parecer um conceito novo, mas a ideia é bastante natural. Afinal, nós já pensamos dessa forma no nosso dia a dia.

É importante fazer uma observação.

A Orientação a Objetos é um assunto bastante amplo. Existem conceitos como classes, objetos, atributos, métodos, herança, encapsulamento e polimorfismo, que normalmente são estudados em profundidade em outros cursos.

O objetivo desta apostila não é explorar todos esses conceitos detalhadamente, mas apresentar uma visão geral para que você compreenda por que esse paradigma é tão utilizado no desenvolvimento de software.

Vamos criar nossa primeira classe.

```
class Carro:
```

```
    def __init__(self, marca, modelo):
```

```
        self.marca = marca
```

```
        self.modelo = modelo
```

Agora podemos criar um objeto.

```
meu_carro = Carro("Chevrolet", "Onix")
```

```
print(meu_carro.marca)
```

Resultado: Chevrolet

Se esse código pareceu um pouco diferente dos anteriores, está tudo bem. Quero apenas que você perceba uma ideia muito importante.

Agora conseguimos representar objetos do mundo real diretamente dentro do

programa.

É exatamente por isso que a Orientação a Objetos é utilizada em praticamente todos os sistemas modernos.

Imagine um aplicativo de banco, onde cada cliente pode ser representado por um objeto.

Cada conta também.

Cada transferência.

Cada cartão.

Essa forma de organizar o código facilita muito o desenvolvimento de projetos grandes.

PROJETO PRÁTICO

Sistema de Controle de Estoque

Objetivo: desenvolver uma aplicação que utilize arquivos, JSON e datas para simular uma situação real encontrada em empresas.

Já aprendemos a manipular arquivos, trabalhar com formatos como TXT, CSV e JSON, utilizar datas e horários, conhecer as expressões regulares, consumir APIs e entender os primeiros conceitos da Orientação a Objetos.

Agora é hora de reunir forças e boa parte desses conhecimentos em uma única aplicação.

Imagine que você foi contratado para desenvolver um pequeno sistema de controle de estoque para uma loja.

O programa deverá permitir o cadastro de produtos e salvar essas informações em um arquivo JSON. Além disso, cada cadastro deverá registrar automaticamente a data e o horário em que foi realizado.

Desafio

Desenvolva um programa capaz de:

- Cadastrar produtos;
- Solicitar nome, preço e quantidade;
- Registrar a data e o horário do cadastro;
- Armazenar os dados em um arquivo JSON;
- Ler o arquivo e exibir todos os produtos cadastrados.

Já sabe, né?

TENTE!

Solução

```
import json

from datetime import datetime

produtos = []

continuar = "S"

while continuar == "S":

    nome = input("Nome do produto: ")
    preco = float(input("Preço: R$ "))
    quantidade = int(input("Quantidade: "))

    produto = {
        "nome": nome,
        "preco": preco,
        "quantidade": quantidade,
        "cadastro": datetime.now().strftime("%d/%m/%Y %H:%M:%S")
    }

    produtos.append(produto)

    continuar = input("Deseja cadastrar outro produto? (S/N): ").upper()

with open("produtos.json", "w", encoding="utf-8") as arquivo:

    json.dump(produtos, arquivo, indent=4, ensure_ascii=False)
```



```
print("\n===== PRODUTOS CADASTRADOS =====")
```

```
with open("produtos.json", "r", encoding="utf-8") as arquivo:
```

```
    lista = json.load(arquivo)
```

```
for produto in lista:
```

```
    print("-----")
```

```
    print("Produto:", produto["nome"])
```

```
    print("Preço: R$", produto["preco"])
```

```
    print("Quantidade:", produto["quantidade"])
```

```
    print("Cadastrado em:", produto["cadastro"])
```

O que aprendemos neste projeto?

Durante este exercício utilizamos diversos conceitos estudados ao longo da trilha.

- Manipulação de arquivos;
- Arquivos JSON;
- Biblioteca datetime;
- Estruturas de repetição;
- Listas;
- Dicionários;
- Leitura e gravação de arquivos.

AGRADECIMENTO

Se você chegou até aqui, quero te dizer: obrigada.

De verdade.

Finalizar um material assim não é só sobre aprender conteúdo. É sobre dedicação, tempo e vontade de evoluir. E só de você estar aqui, já mostra muito sobre você.

Essa apostila foi criada com um propósito muito claro: mostrar que conhecimento não precisa ser complicado. Que assuntos que parecem difíceis podem, sim, ser entendidos quando explicados da forma certa.

E mais do que isso: que conhecimento deve ser para todos.

Por isso, este material também foi pensado para ser acessível. Ele conta com versão em Braille, porque inclusão não é um diferencial — é o mínimo.

Todo mundo merece aprender. Todo mundo merece ter acesso.

Se, de alguma forma, esse conteúdo te ajudou a entender melhor, a se organizar ou até a enxergar as coisas de um jeito diferente, então ele já cumpriu o seu papel.

Obrigada por fazer parte disso.

E lembre-se: você não precisa complicar para fazer dar certo.

CONTATO

Se você quiser continuar aprendendo, trocar ideias ou acompanhar mais conteúdos sobre projetos, tecnologia e organização no dia a dia, você pode me encontrar por aqui:

Instagram: @CAROL_G_INTECH

LinkedIn: Caroline Gonçalves

Site: <https://cpgconsulting.com.br/>

E-mail: cpgtechconsulting@gmail.com

Fique à vontade para se conectar ou mandar uma mensagem. Vou adorar continuar essa troca com você.